Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Automated Threat Hunting

Learn more at Stellar Cyber Academy.

The following link takes you to a course on the Stellar Cyber Academy technical training portal where you can learn more about this topic by watching the suggested lessons.

J03 Investigations and Workflows (05h:48m)

Workflow Tools in the Stellar Cyber UI (35m:00s)

Explore workflow tools within the Stellar Cyber UI, including case management, knowledge sharing, task assignment, and alert triage. Discover ways to enhance collaboration, automate notifications, and integrate the UI tools into efficient SOC workflows.

Building Workflows in Stellar Cyber - Foundations (45m:00s)

Discover foundational strategies for building workflows in the Stellar Cyber Platform, focusing on tools that support knowledge transfer, case management, and task tracking. Learn how to structure workflows to streamline operations within your SOC.

The first time you access a link on the portal during a session, you must log in to access content.

Automated Threat Hunting (ATH), accessible from the **Automation | Automation** menu, is a key aspect of the Stellar Cyber architecture to enable automated responses, creation of data, and other actions. Stellar Cyber automatically notifies you by whatever method you configure when it finds a possible threat. The system includes more than 250 playbook templates and Stellar Cyber is always developing more.

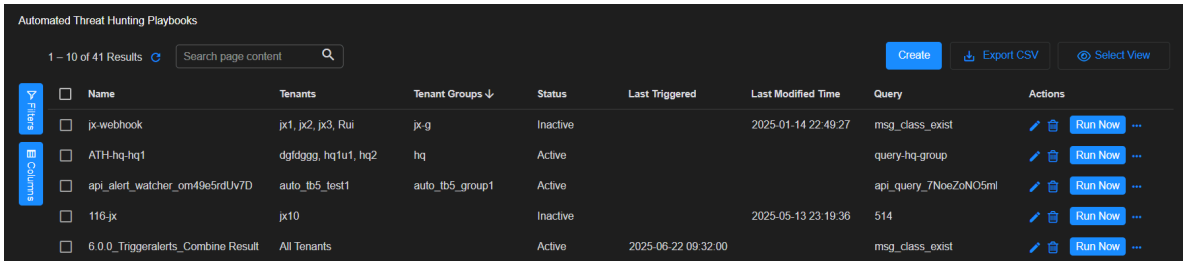
This article is organized into the following main sections:

- **ATH Playbook List**
- **Create or Edit a Playbook**
- **Trace an Alert to Its Underlying Event Records**
- **About Deduplication of Custom Alerts**

ATH Playbook List

When you navigate to **Automat| Automation**, Stellar Cyber displays the Automated Threat Hunting playbook list. The list begins with two default playbooks and includes any that you create or load from a template.

The visible ATH playbooks are restricted to those accessible to the currently active tenant. The columns for **Tenant** and **Tenant Group** are displayed only for Partner users and Root users.



ATH Playbook Columns

Two of the columns in the ATH Playbook table are **Last Checked** and **Last Triggered**. It's important to have a clear understanding about what each one means.

Last Checked – Shows the last time Stellar Cyber executed the ATH playbook query to check for matching events. This value updates only when the playbook is active and within its scheduled run window.

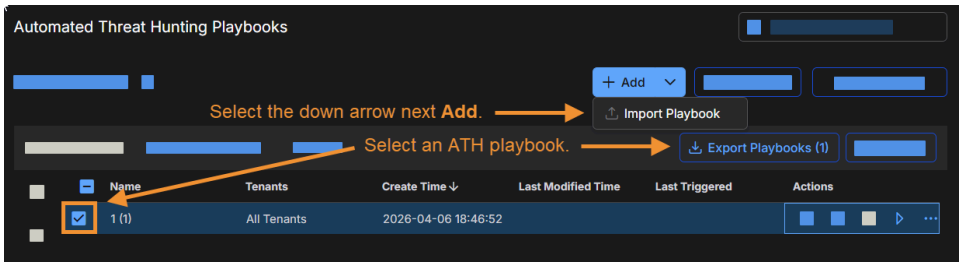
Last Triggered – Shows the last time the ATH playbook found matching events and triggered its configured action.

Playbook Management

You can create a new playbook using the following techniques:

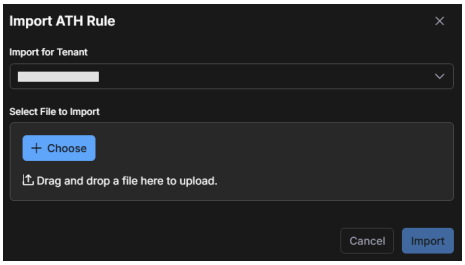
- **Creating**: To open editor with blank fields and define a completely new playbook, select the **Create** button.
- **Cloning**: To use an existing playbook as a template, select one or more check boxes at the start of a row and then click the **Clone** button. The playbook(s) are duplicated and named the same as the original with a timestamp added. Edit the copy to customize it, as needed.
- **From a Custom Alert Type in the Alerts Page**: As described in Creating New Alert Types , you can use a custom alert type **Add Playbook** option in the Alerts list to create a new playbook pre-filled with the settings from the custom alert type.

You can move ATH playbooks between environments by importing and exporting them.



- **Exporting**: To export one or more playbooks, select the check boxes at the start of the corresponding rows and then select **Export Playbooks**. Stellar Cyber downloads the selected playbooks as an export file that you can save locally for backup, reuse, or transfer to another deployment.
- **Importing**: To import previously exported playbooks, select the down arrow next to **+ Add**, select **Import Playbook**, and then choose the previously exported file. Stellar Cyber validates the file and imports the playbooks into the current environment.

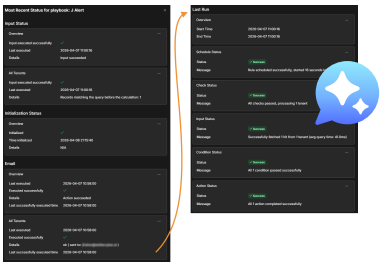
  Only ATH playbooks that have the action set as **Create Alert** can be imported.

  An imported ATH playbook includes dependent components, such as queries and custom alert types. If these components don't already exist in the current environment, Stellar Cyber adds them during import. If they do already exist, Stellar Cyber skips them.
- **Name Conflict During Import**: If an imported playbook has the same name as an existing playbook, Stellar Cyber adds the new playbook with a counter appended to its name; for example, “Playbook” > “Playbook (1)”.
- **Imported Playbooks**: After import completes, review the imported playbooks to confirm that the tenant assignment, query logic, schedule, and configured actions are correct for the target environment before enabling them.

For each playbook in the list, you can do the following:

- Select to edit the playbook. The configurations in the **Edit** dialog box are identical to those in the **Add** dialog box.
- Select to delete the playbook. You must confirm the deletion. (You cannot delete the default playbooks.)
- Select **Run Now** to run the playbook immediately, regardless of the schedule, and take any configured actions. You can do this whether the playbook is active or not. Running the playbook now does not reset the schedule.
- Select ••• and **Disable** or **Enable** to immediately disable or enable the playbook.
- Select ••• and **Last Status** to see the **Most Recent Status for Playbook: <playbook-name>** pop-up panel with the results from the last time the playbook ran. The panel includes several sections that show information about the input and initialization of the playbook and the action it performed.



Input Status: This section shows whether the playbook query ran successfully and how many records matched the query.

- **Overview** – Displays whether the query executed successfully and the time it last ran.
- **Tenant** – Displays the tenant assigned to the playbook.

  For playbooks created for all tenants, the section is labeled **All Tenants**.

  For single-tenant playbooks, the section name matches that of the tenant.

  For tenant-group playbooks, the section name shows the individual tenant name if the group contains only one tenant, or **multiple-tenants** if the group includes more than one.

  The **Details** field displays the number of records returned by the playbook query. The message shown depends on whether the query includes a calculation.

  - If the query doesn't include a calculation, the field displays **Query hits count: <number>**, representing the total records that matched the query.
  - If the query includes a calculation, the field displays **Records matching the query before the calculation: <number>**, representing the raw query results before the calculation is applied.

  When a calculation is present, records might match the base query but not generate alerts if the calculation doesn't return a qualifying result. In this case, the pre-calculation count can be greater than zero even though no alerts are produced.

Initialization Status: This section shows whether the playbook initialized successfully before execution. Initialization confirms that the playbook configuration is valid and that all required components are available.

- **Overview** – Displays the initialization status, the time initialized, and any relevant details.

Action: This section shows the results of the action that the playbook triggered when its criteria were met. The section title reflects the action type—for example, **Email**, **Webhook Responder**, and so on.

- **Overview** – Lists whether the action executed successfully and the time it last ran.
- **Tenant** – Displays the tenant assigned to the playbook, labeled **All Tenants**, **multiple-tenants**, or with an individual tenant name. The **Details** field lists the

action result, such as the recipient's email address or the webhook endpoint.

Last Scheduled Run: This section shows the results of the most recent scheduled run attempt for the playbook, whether or not the playbook query or action ran successfully.

Last Manual Run: This section shows the results of the most recent manual run attempt. It appears when you have run the playbook at least once by selecting **Run Now**.

The following sections are in both **Last Scheduled Run** and **Last Manual Run**:

- **Overview** – Displays the start time and end time of the most recent scheduled run attempt.
- **Schedule Status** – Displays whether the playbook was scheduled successfully and any related scheduling message.
- **Check Status** – Displays whether the playbook passed the checks required to run, along with any related message or errors. For example, this section can show that the rule did not run because it was inactive.
- **Input Status** – Displays whether the playbook successfully retrieved the input data required for evaluation and any related message.
- **Condition Status** – Displays whether the playbook condition evaluated successfully and any related message or errors.
- **Action Status** – Displays whether the playbook action executed successfully and any related message or errors.

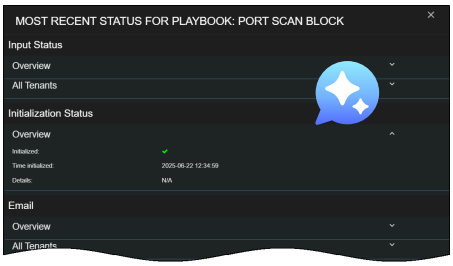
If the playbook runs on schedule but its condition is not met, the **Action** section does not update. In this case, it continues to display the results from the last time the playbook successfully triggered an action. For example, if the rule last triggered an alert yesterday but has since run several times without meeting its condition, the **Last executed** field in this section continues to show yesterday's date. The **Input Status** section, however, updates each time the playbook runs.

When Stellar Cyber sends an email message with a file attachment that's too large for email servers to forward, it displays the following message when you select **… | Last Status** for one of the Automated Threat Hunting playbooks:

```
Attachment too large (Email server returned 500, message: None), sent without interflow attachment (sent to: user@domain.name).
```

In this case, the **Executed successfully** field shows a red **X** to indicate no attachment was included.

For newly created playbooks, a basic validation is performed when you first save the configuration and when the playbook runs. The result of this validation is displayed in the **Initialization Status** section of the **Most Recent Status for Playbook: <playbook-name>** panel.



Default Playbooks

Stellar Cyber includes default playbooks for common threat hunts so you can focus on your custom threat hunts. The default playbooks have a creator of **System**.

You can edit the default playbooks to add recipients, actions, or anything else. You can disable them, but you cannot delete them. The default playbooks are:

- Data Ingestion Anomaly for Network and Security Sensors
    - **Description** – a data ingestion anomaly has been reported at least once
    - **Frequency** – every 5 minutes
    - **Indices** – aella-ser-*
    - **event_name** – ade_outbytes_anomaly
    - **Action** – email to every user with the Super Admin profile
- Data Ingestion Anomaly for Agent Sensors
    - **Description** – a data ingestion anomaly has been reported at least once
    - **Frequency** – every 5 minutes
    - **Indices** – aella-ser-*
    - **event_name** – ade_outbytes_anomaly
    - **Action** – email to every user with the Super Admin profile

Create or Edit a Playbook

When you select the **Create** button from the **Automation | Automation** playbook list, or from an **Alerts** table, a fresh playbook editor appears. If you edit a playbook, the same editor appears with its previously configured settings. The steps are the same, whether you are adding or editing and whether you opened it from the **Automation | Automation** or **Detections | Alerts** menu.

The configuration of an ATH playbook involves the following steps:

Objectives	Steps	Links
1 – Set the foundation	Define the playbook name, schedule, tenant, and index in the **Alert Configuration** section.	See **Alert Configuration**.
2 – Build detection logic	Create or load your query or correlation in the **Query** section.	See **Build a Query** or **Build a Correlation**.
3 – Refine query results	Optionally perform mathematical operations and aggregate data returned in your query results in the **Calculations** section.	See **Add Calculations (Optional)**.

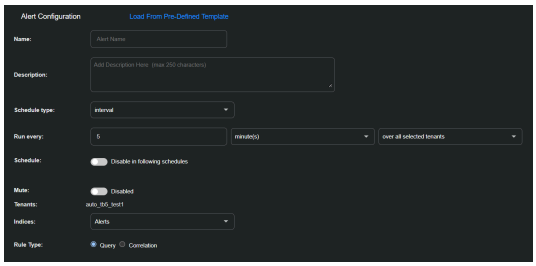
4 – Specify trigger conditions	Define your logic for triggering alerts in the **Condition Configuration** section.	See **Add Conditions**.
5 – Configure responses	Set up automated responses in the **Actions** section.	See **Configure Actions**.
6 – Submit your playbook	Save and activate the configuration. The playbook is automatically enabled.	

Use the remainder of this section as a guide for how to work with each section of the playbook page. For specific examples, refer to the following descriptions:

- Create a playbook that uses nested calculations to block a port-scanning source IP address.
- Create a playbook that looks for sensors that haven't transmitted data for a while.
- Create a playbook that looks for too many half-opened sessions.
- Create a playbook that looks for pass-the-hash attacks.
- Create a playbook that sends an email or Slack message whenever a case's score exceeds a specified threshold.
- Create a playbook that resets TCP connections after detecting threats.

Alert Configuration

The **Alert Configuration** section is at the top of the ATH playbook.



Stellar Cyber recommends using one of the templates to start. You can modify the template as much as you want.

1. Select **Load From Pre-Defined Template**.
2. Select a template.
3. Select **Submit**.

   The playbook is immediately populated with the template settings.
4. Enter a name for the playbook.

   The name is automatically the template you loaded; you can change it.
5. (Optional) In the **Description** field, enter a note about the purpose of the playbook for future reference.

   You can enter up to 250 characters in any language, including characters in unicode.
6. Select the **Schedule** type.

   You can select anything from **interval** (to specify minutes) to **Monthly**.

Scheduled times entered as part of **Daily**, **Weekly**, and **Monthly** schedules are expressed in UTC not the timezone of your browser.

7. Use the **Run every** options to specify when you want the playbook to run given the type of schedule you selected in the previous step. For example, if you chose **Monthly** in the previous step, you can use the **Run every** options to specify on which day of the month to run the playbook.

   In previous releases, if you chose **Per Tenant**, you could only send an email as the action. This restriction is no longer in place – you can now use either the **Email** or **Create Alert** action when **Per Tenant** is selected.

8. Use the optional **Schedule** settings to disable the playbook during specific time windows.

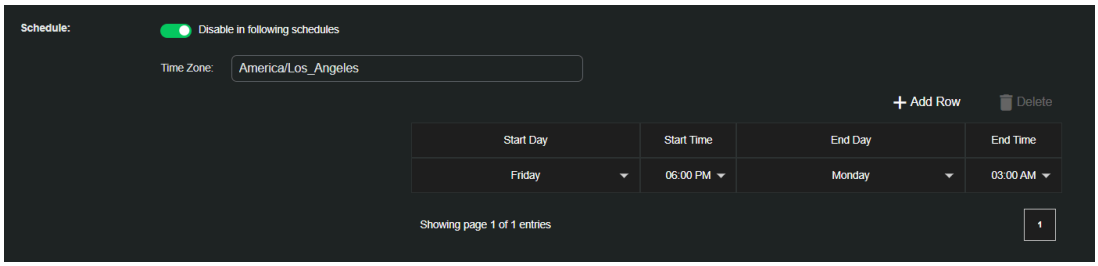
   For example, you could restrict the playbook to running only during business hours in Europe.

   a. Select **Disable in following schedules** to turn the option on.

   b. Choose the time zone from the dropdown list.

   c. Use the table that appears to schedule windows of time where the playbook does not run.

      For example, in the figure below, weekends are excluded:



      If necessary, delete rows by selecting them and then selecting **Delete**.

      If you have trouble getting the schedule rows to display the available options, try double-clicking.

9. To temporarily silence alert generation from the playbook for a specified amount of time, toggle **Mute** from **Disabled** to **Enabled** and use the adjacent fields to specify for how long you want to stop triggering alerts.

   You can also mute individual actions associated with the playbook. Note that Alert-level mutes take precedence over Action-level mutes. If the alert is muted, no actions will be triggered. If the alert is unmuted, you can mute individual actions.

10. Select the tenants or tenant groups on which to run.

    Sending an alert to a tenant group has the same performance impact as sending alerts to each individual tenant.

    If you configure an Automated Threat Hunting (ATH) playbook to create alerts using the **Create Alert** action and you add calculations, set the scope to a single tenant rather

than **All Tenants**. Alerts cannot be created if calculations are applied across multiple tenants because the results cannot be associated with a specific tenant. Other actions, such as email notifications, Slack messages, webhook queries and so on, still work because they don't require a specific tenant ID.

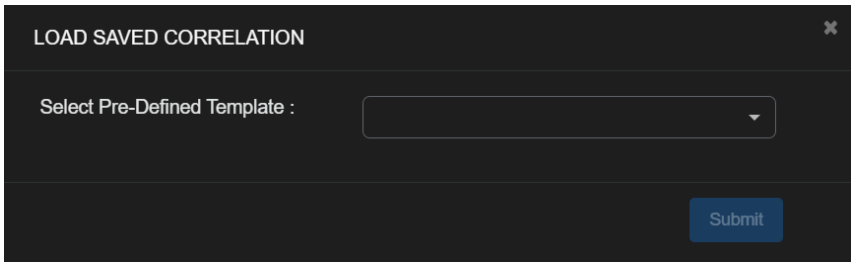
11. Select the index.
12. Select the rule type.

    The default for this template is **Query**. If you select **Correlation**, you can load a correlation or create a new one.

Build a Correlation

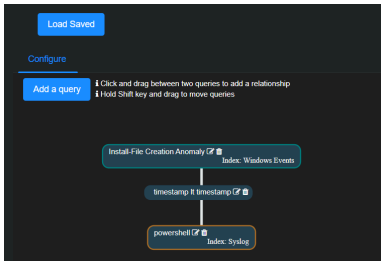
Correlation settings appear if you select **Correlation** as the rule type.

To load a saved correlation, select **Load Saved**, choose the saved correlation, and then **Submit** your choice.

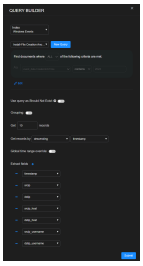


Your correlation is immediately displayed.

However, for this correlation example, you create your own correlation.



1. Select **Add a query** and add two queries
2. Click the ✎ button on each query to edit its details.



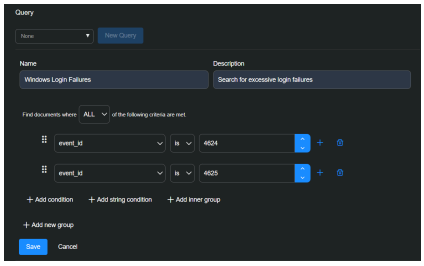
> When you create a correlation for use in a Firewall action, ensure that **Grouping** is not enabled. In addition, the list of **Extracted Fields** (see above image) must include `srcip` and `dstip` in the query object for the host on which you want to perform the Firewall action. That same query object must be enabled with the Active setting when you perform the Firewall action configuration, below.

3. Add a relationship.

4. Continue with the rest of the playbook configuration.

Build a Query

The **Build a Query** section appears if you select the **Query** rule type. This is the query you want to run on the specified tenant.



The template built a query, which can be changed if you want. To edit (or build) your query:

1. Enter a name and an optional description for the query.

   The name is automatically that of the template you loaded. The query name is listed in the Automated Threat Hunting Playbooks list for easy reference.

   > Special characters are not permitted in name fields for queries, watchlists, reports, or dashboards. Letters, underscores, spaces, dashes, numbers and periods are permitted.

2. Build your query.

   When referencing fields in a query condition, use `_source`, such as `_source.appid_name`. This is required because ATH queries operate on Elasticsearch results where each document is wrapped in a `_source` object under `hits.hits[]`. In contrast, when specifying a field in a calculation, use only the field name—for example, `appid_name`. This is because calculations are based on aggregation buckets located in `aggregations.<name>.buckets[]`, where fields are not nested under `_source`. While the UI displays field names without `_source.` due to internal optimization, this does not affect how queries or calculations must be written.

   > The scope of a query determines which tenants can use it. Whether it's possible to create a query for **All Tenants** depends on the scope of the object being queried, such as charts, correlations, and Automated Threat-Hunting (ATH) rules. In short, the scope of a queried object cannot be more restrictive than the scope of the query itself. For example, if you create an ATH rule for **All Tenants**, then the query for this ATH rule can either be **All Tenants** or just a single tenant, such as "Tenant A" for example. However, if you create an ATH rule for Tenant A, then the query cannot be for **All Tenants** because the other tenants won't have this ATH rule and won't be able to query it. In this case, the query can only be for Tenant A.

   Query Execution and Result Limits

   ATH playbooks run on a fixed schedule or interval. Each execution runs a predefined query against the data lake. After retrieving results, the playbook checks whether the results meet specified match conditions, such as the presence of a threshold number of specific events. These events can include alerts, traffic logs, syslog records,

Windows event logs, or other data indexed in the data lake. If the results match a condition, the playbook proceeds to execute the defined automated response.

After the scheduled execution begins, the query engine searches the data lake using the defined search criteria. It looks for Interflow records that match the specified conditions and returns results relevant to the detection objective of the playbook. For example, a query might search for "new login locations by users within the last 24 hours" if an unusual login alert triggered the playbook.

To manage performance and manage resource usage, the Stellar Cyber Platform enforces query result limits for ATH playbooks based on how each playbook scopes tenants. When a playbook runs over all selected tenants, it limits the total returned results to a global maximum of 5,000 Interflow records across all tenants. When it runs per tenant, it retrieves no more than 100 records per tenant. These limits help maintain consistent performance regardless of the execution scope.



You can contact Customer Success and request an increase to the maximum number of records returned per tenant from 100 up to 500. However, increasing this maximum can might introduce instability to the ATH rule and latency to the Stellar Cyber Platform.

These thresholds apply only to the records returned in the query results. They do not affect total match counts. Stellar Cyber calculates metrics such as total hits or alert logic using all matching records, even if the query returns only a portion..

The query engine always enforces the per-tenant 100-record limit when it runs a playbook for an individual tenant—whether the tenant creates the playbook directly or receives it through a deployment from a partner or root-level administrator. When a partner or root-level administrator deploys a single ATH playbook across multiple tenants and those tenants run the playbook at the same time, the engine first applies the 5,000-record global limit. It sorts all matching records by descending timestamp and selects the top 5,000 records. From that global set, it returns up to 100 records per tenant.

When you configure a playbook to run **over all selected tenants**, the query engine first sorts all matching records by descending timestamp and selects the top 5,000 records across all tenants. When you configure the playbook to run **per tenant**, it still applies the global 5,000-record cap first. Within that global set, the engine returns up to 100 records per tenant. If a tenant has more than 100 records in the top 5,000, the engine includes only the first 100. If a tenant has no records in the top 5,000, it is skipped entirely for that execution cycle.

When tenants run the same playbook at different times, Stellar Cyber treats each execution separately. In this case, the engine applies only the 100-records-per-tenant limit to each execution and does not carry forward or accumulate the global limit across time.

If you assign a playbook to **All Tenants** and configure it to run **over all selected tenants**, the engine executes a single query and applies the global maximum of 5,000 Interflow records. In this mode, the per-tenant limit is not applied—only the

global maximum controls the results. If you instead assign a playbook to **All Tenants** and configure it to r**er tenant**, the engine runs the query separately for each tenant. It applies the 100-record per-tenant limit within the first 5,000 records retrieved globally across all executions. Once the global limit is reached, the platform stops processing additional tenants for that execution cycle.

3. Save the query.

From this point you can take the following actions:

- Select the saved query name to load another saved query.
- Select **Edit** to return to query-builder mode.
- Select **New Query** to build another query.

  For more details on Queries, see **Queries and Filters**.

- Select **Calculations** to perform data aggregation and analysis within query results in the playbook. (If you add a calculation, you can include a CSV file with an email action).

  For another example using multiple nested calculations, see **ATH Example: Blocking a Port Scanning Source IP** .

Add Calculations (Optional)

The **Calculations** section in an Automated Threat Hunting (ATH) playbook is an optional feature that's disabled by default. To use it, you must first enable the section. After that, its configuration options become visible. Calculations allow you to perform data aggregation and analysis directly within query results before conditions and actions are applied. This helps to prioritize threats, reduce noise, and improve the accuracy of automated detection.

When adding calculations to an ATH playbook, ensure that the scope is set to a single tenant if you intend to use the **Create Alert** action. If the scope is set to **All Tenants** and multiple tenants are involved, alerts cannot be created because calculation results span multiple tenants and Stellar Cyber cannot assign a valid tenant ID to the alert. Other actions continue to function because they don't depend on a tenant ID.

The **Calculations** section supports different types of data aggregations, including metric aggregations (such as sum or average) and bucket aggregations (such as top or bottom-ranked values). When a calculation is applied, its result can be referenced in the **Conditions** section of the playbook, allowing you to create more refined automation logic. For example, a calculation could determine the top 5 source IP addresses by event count, and a condition could then trigger actions only if the count exceeds a threshold.

Naming Calculations

When you name a calculation, do not use names that duplicate common system field names. For ATH playbooks that use the **Create Alert** action, the calculation name becomes the field name for the calculation result in the generated alert, so it must not conflict with reserved or commonly used field names. For example, instead of naming a calculation **username** (which is a common field name), you could use something like **username_count**.

Calculation names must use only letters, digits, and underscores, and they cannot start with a digit. These naming rules prevent ATH calculations from creating invalid field names in generated alerts. Existing calculations that do not follow the current naming rules continue to run after upgrade, but if you edit one, you must rename it before saving.



These naming requirements apply to Stellar Cyber 6.6.0 and later, and to 6.4.0 and 6.5.0 patch releases issued after 6/18/2026.

Calculation Types

The **Calculations** section in an ATH playbook allows users to analyze data using three types of calculations:

- **Top**: Retrieves the highest-ranking values based on a selected field
- **Bottom**: Retrieves the lowest-ranking values based on a selected field
- **Metric**: Performs a mathematical operation on a dataset

When you select **Top** or **Bottom**, you must configure these settings:

- **Field to calculate on**: The dataset field used for ranking

  When specifying a field in **Field to calculate on**, use only the field name—for example, `appid_name`—and not `_source.appid_name`. This is because calculations are based on aggregation buckets located in `aggregations.<name>.buckets[]` and fields aren't nested under `_source`. In contrast, when referencing fields in a query condition, do use `_source`, such as `_source.appid_name`. This is required because ATH queries operate on Elasticsearch results where each document is wrapped in a `_source` object under `hits.hits[]`. While the UI displays field names without `_source.` due to internal optimization, this does not affect how queries or calculations must be written.

  If using a text field, you must append `.keyword` to the field name.

- **Calculate By**: Defines the mathematical operation (see Section 2)
- **Size**: Specifies the number of results to return (default: **10**)

When you select **Metric**, you must choose two components for the calculation from two dropdown lists:

- From the first list, choose one of several mathematical operations to perform on the dataset: average, maximum, minimum, sum, or unique count.
- From the second list, choose a field on which to apply the operation.

Mathematical Operations

Each calculation type supports different mathematical operations



If you choose a mathematical operation other than Count, you must also choose the field on which to perform the operation.

Count: Counts the total number of aggregated Interflow records that include the specified field and therefore fall into the same bucket, or group, as it's called in the Stellar Cyber UI. Examples:

- Counting the number of alerts for a given IP address
- Counting the total number of failed login attempts across all users
- Counting the number of times a specific file is accessed

Average: Calculates the average of numeric values extracted from the specified field in all aggregated Interflow records. Examples:

- Determining the average response time for HTTP requests
- Calculating the average number of connections made per user
- Finding the average CPU utilization across multiple servers

Maximum: Returns the highest value extracted from the specified field in all aggregated Interflow records. Examples:

- Finding the maximum number of login attempts from an IP address
- Identifying the peak network bandwidth usage in a given timeframe
- Determining the highest number of alerts generated for a single user

Minimum: Returns the lowest value extracted from the specified field in all aggregated Interflow records. Examples:

- Identifying the shortest session duration for a user
- Finding the lowest CPU utilization recorded in the last 24 hours
- Determining the smallest number of login attempts made by an individual user

Sum: Adds up numeric values extracted from the specified field in all aggregated Interflow records. Examples:

- Summing up the total number of bytes transferred by a source IP
- Summing the total duration of all remote desktop sessions in a given period
- Summing the total number of login attempts across all users

Unique Count (cardinality): Calculates an approximate count of distinct values extracted from the specified field in all aggregated Interflow records and ignores records with duplicate values in this field. Examples:

- Counting the number of unique users accessing a particular server
- Counting the distinct IP addresses that have connected to a network in the last 24 hours
- Counting the number of unique domains visited by a specific user

Nested Subgroups

Subgroups in Automated Threat Hunting (ATH) calculations let you drill deeper into data relationships by creating hierarchical layers of analysis. They refine the primary Top or Bottom calculation by applying additional filters within its results. This helps you identify attack patterns, prioritize threats, and automate detection with greater accuracy.

For example, instead of simply identifying the top 10 most active source IP addresses in failed login attempts, you can add a subgroup to further break down the top 5 destination hosts targeted by each of those addresses. You can then add another subgroup to list the top 3

usernames used in the failed logins for each host. By layering calculations like this, you can more easily detect patterns such as credential stuffing attacks or lateral movement.

Understanding the display of subgroups in the UI

Each subgroup is nested beneath its parent calculation, forming a tiered structure that lets you refine results step by step. The UI visually represents these relationships using indentation and L-shaped lines that connect each subgroup to its parent.

- **Parent Calculation** (top-level calculation): The first calculation that defines the dataset scope
- **Subgroup** (child calculation): A calculation applied within the results of the parent calculation
- **Additional Subgroups**: Each subsequent subgroup filters data from its immediate parent, creating deeper levels of segmentation.

Example: Tracing Failed Login Attempts

This example demonstrates how to use nested subgroups to investigate failed login attempts, a key indicator of brute-force attacks or credential stuffing. The following are the three goals of these calculations:

- Identify the top 10 most active source IP addresses that are generating failed login attempts.
- Within each of these addresses, find the top 5 destination hosts they are targeting.
- For each host, determine the top 3 usernames used in failed login attempts.

By structuring calculations in this way, you can correlate attack behavior across different dimensions and focus your security response on the most targeted systems and compromised credentials.



The Query in this ATH playbook filters Interflow records that match the condition: `event_name is user_login_fail`.

1. Enable the **Calculations** section in your ATH playbook.
2. To create the parent calculation, select the **+** (**Add a Calculation**) icon next to **Calculations** to add a new calculation and enter the the following:

   Name: `Top 10 Source IPs for Failed Logins`

   Calculation type: **Top**

   Field to calculate on: **srcip**

   Calculate By: **count**

   Size: **10**
3. On the right side of the newly created `Top 10 Most Active Source IPs` calculation, select **Add Subgroup** and then enter the following:

   Name: `Top 5 Destination Hosts`

   Calculation type: **Top**

Field to calculate on: **host.external_ip**

Calculate By: **count**

Size: **5**

4. On the right side of the `Top 5 Destination Hosts` calculation, select **Add Subgroup** and then enter the following:

   Name: `Top 3 Usernames in Failed Logins`

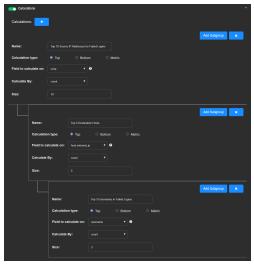
   Calculation type: **Top**

   Field to calculate on: **username**

   Calculate By: **count**

   Size: **3**

After adding these calculations, Stellar Cyber displays nested calculations with each subgroup appearing indented beneath its parent.

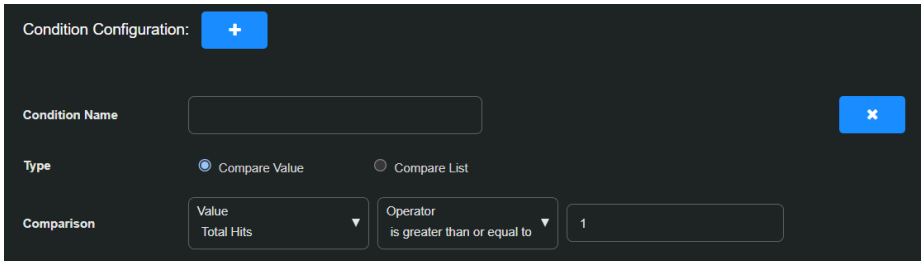


Each subgroup calculation only applies within the results of its parent group, letting you refine security analysis step by step.

Add Conditions

Automated Threat Hunting (ATH) playbooks let you define conditional logic that determines when an action should be executed. Conditions are configured within the **Condition Configuration** section of a playbook and can be based on direct field values or calculated results from an aggregation, which is a grouping of Interflow records based on specified fields.

After configuring settings in the **Alert Configuration**, **Query**, and (optionally) **Calculations** sections, select **+** (**Add a Condition**) in the **Condition Configuration** section.



Condition Types

When configuring a condition, the **Type** setting determines how the condition evaluates data. The following are the two **Type** options:

- **Compare Value**: Directly compare a field value against a comparison value.
- **Compare List**: Evaluate whether one or more values in a dataset match a condition.

Compare Value

When you select **Compare Value**, the **Value** setting presents a single option: **Total Hits**. This means the comparison applies to the total number of Interflow records matching the query defined earlier in the playbook. After choosing an operator from the list (`does not equal | equals | is greater than | is greater than or equal to | is in the range | is less than | is less than or equal to`), enter a comparison value in the field on the right.



The `is in the range` operator evaluates numeric fields inclusively. Both the lower and upper boundary values are included in the result.

Example: Detecting Excessive Login Failures

This condition triggers when a query defined earlier in the playbook gets results for at least five failed login attempts within a monitored dataset, which might indicate a brute-force attack or unauthorized access attempts.

- **Condition Name**: `Multiple Failed Logins`
- **Type**: **Compare Value**
- **Value**: **Total Hits**
- **Operator**: **is greater than or equal to**
- **Comparison Value**: **5**

Compare List

When you select **Compare List**, the **Field** setting provides multiple options depending on whether a calculation has been configured within the playbook.

Comparison Field Options

The **Field** list includes options from two sources:

1. **Calculation-Based Fields**

   If a calculation is configured in the **Calculations** section of the playbook (above the **Conditions** section), the following options appear:

   - **Result list of** <calculation-name>
   - **Result counts of** <calculation-name>

   These options allow the condition to compare against the computed results of the configured calculation. The calculation name in these options corresponds to the name assigned in the **Calculations** section of the playbook.

   Configuring a calculation is optional. If no calculation has been set, these options don't appear in the **Field** list, although you can still choose among the security alert field names that comprise the rest of the items in the **Field** list.

2. **Event-Based Fields from Security Alert Interflow Records**

   The Field list contains a broad set of security alert metadata field names extracted from Interflow records. These field names originate from various security alerts generated by Stellar Cyber or detected through third-party security integrations and can include attributes such as these for example:

- **abnormal_security.attackVector** – The method used in the attack (for example, phishing, malware, and exploit).
- **abnormal_security.attachmentCount** – The number of attachments in an email security event.
- **abnormal_security.category** – The classification of the alert (for example, phishing, impersonation, and data exfiltration).

These field names allow conditions to be based on **raw event data** independent of any configured calculations.

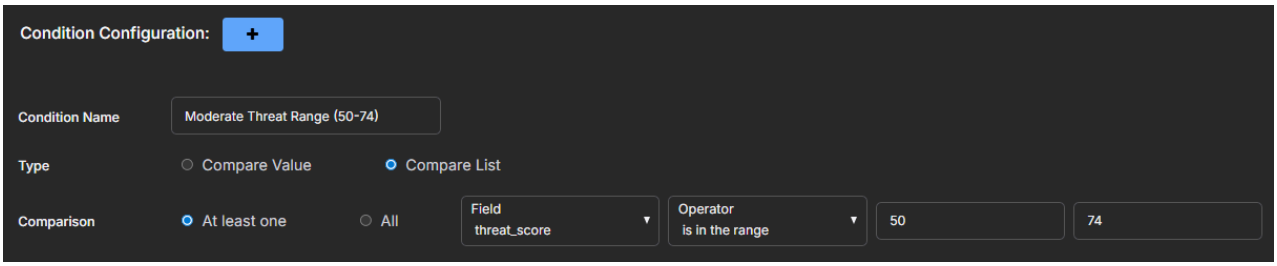
Example: Identifying Anomalous Threat Scores Within a Range

This condition triggers when a query defined earlier in the playbook returns results where at least one record includes a Threat Score value that falls between 50 and 74 (inclusive). This configuration identifies moderate-risk activities that are below the critical threshold but still warrant analyst review.

- **Condition Name**: **Moderate Threat Range (50–74)**
- **Type**: **Compare List**
- **Comparison**: **At least one**
- **Field**: **threat_score**
- **Operator**: **is in the range**
- **Comparison Value**: **50–74**

> Using **Compare List** with **At least one** triggers the condition if any record meets the range criteria. Selecting **All** instead requires every record to fall within the range; if even one record is outside it, the condition fails, and no actions are executed.

This configuration allows playbooks to isolate events that fall within a specific risk band, giving you greater control over how intermediate threat levels are automatically evaluated and acted upon.



Example: Identifying Large Data Transfers

This condition is designed to detect excessive outbound data transfers, which might indicate data exfiltration, unauthorized file transfers, or insider threats. It ensures that any host exceeding a 1-GB data transfer threshold is flagged for security review.

The condition references a previously defined calculation (High-Transfer-Hosts), which identifies the top 10 hosts with the highest outbound data volume. The condition references a previously defined calculation (High-Transfer-Hosts), which identifies the top 10 hosts with the highest outbound data volume. By applying a threshold of 1,000,000,000 bytes (1 GB), this

rule isolates only significant data transfers, reducing false positives from routine network activity.

- **Condition Name**: **Data Transfer Threshold Exceeded**
- **Type**: **Compare List**
- **Comparison**: **At least one**
- **Field**: **Result list of High-Transfer-Hosts**
- **Operator**: **is greater than or equal to**
- **Comparison Value**: **1000000000**

Comparison Modes

When configuring a **Compare List** condition, the **Comparison** setting affects how results are evaluated:

- **At least one**: The condition is met if at least one result matches the criteria.
- **All**: The condition is met only if all results match the criteria.

These options are only available when using **Compare List**. If **Compare Value** is selected, the **Comparison** setting is removed.

For example, suppose a calculation returns these values: 2, 7, and 9.

If the condition checks whether the value is **greater than or equal to 5**:

- **At least one** passes because at least one value in the list matches the rule. In this example, 7 and 9 match the rule.
- **All** fails because not every value in the list matches the rule. In this example, 2 does not match the rule.

Best Practices

- Use **Compare Value** when working with overall counts.
- Use **Compare List** when evaluating specific field values or calculated results.
- Ensure the correct **Comparison** mode is selected based on whether any or all results must match.
- If using calculations, verify that **Result List** and **Result Counts** appear in the field options.
- If comparing security alert metadata, check Interflow records in **Detections | Alerts** to confirm field values.

Understanding Compare List Results

A **Compare List** condition checks a list of values and determines whether those values meet a rule. The values in a list can come directly from event field values or from calculation results.

If the playbook uses calculations, the returned results are often organized into groups of results. In the underlying query system, each group is called a bucket. You can think of a bucket simply as a group of matching results.

For example, suppose the query finds these failed login events:

- Event 1: source IP `10.10.10.1`
- Event 2: source IP `10.10.10.1`
- Event 3: source IP `10.10.10.2`
- Event 4: source IP `10.10.10.3`
- Event 5: source IP `10.10.10.3`
- Event 6: source IP `10.10.10.3`

If a calculation groups these failed login events by source IP address, the results are organized into these groups:

- Group for `10.10.10.1` → 2 failed login events
- Group for `10.10.10.2` → 1 failed login event
- Group for `10.10.10.3` → 3 failed login events

Each of these groups is called a **bucket**. When Stellar Cyber checks a Compare List condition, it looks at the returned values and sees whether they match your rule. If the condition does not pass, the status can help explain why. In these messages, **path** refers to the specific returned calculation result that the condition is trying to evaluate.

- `Condition path not found: ctx.payload.aggregations....`

  This means the condition could not find the selected calculation result. Check that the referenced calculation or subgroup still exists and that the condition is pointing to the correct result.

- `Condition path exists but contains no buckets`

  This means the calculation result was found, but it did not return any groups of results for this playbook run.

- `No bucket values matched eq 0 (quantifier=some)`

  This means the condition found values to evaluate, but none of them matched the comparison rule.

- `Matched X/Y ...`

  This means one or more values matched the comparison rule. The numbers show how many groups of results matched out of the total number evaluated.

Configure Actions

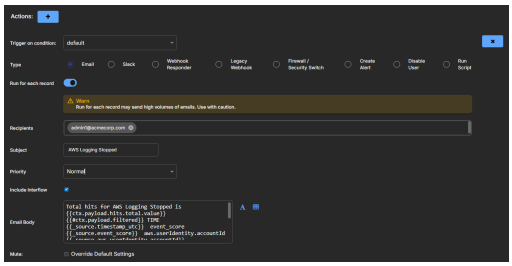
You can configure any number of actions to take place if your playbook gets the specified number of hits. Those actions are:

- Email
- Slack
- Webhook Responder
- Legacy Webhook
- Firewall / Security Switch
- Create Alert
- Disable User

- Run Script

You can also temporarily mute [illegible]ction if you find that you are receiving too much "noise" from it.

Email Action



Sending an email is the default action, so Stellar Cyber automatically selects it and populates the **Subject** and **Email Body**. To send an email as an action:

1. Select the **Trigger on condition**.
2. Ensure that **Email** is selected for the **Type**.
3. If you want the playbook to send a separate email for each matching record, enable **Run for each record**.

   The **Run for each record** option is available only for playbooks that do not use calculations.

   When this option is enabled, the playbook evaluates each result individually and sends a separate email for each matching record.

   Use **Run for each record** carefully. Sending individual emails for each matching record can increase processing time and generate a large number of messages. To reduce message volume, refine the query or apply filtering earlier in the playbook configuration.
4. Select **Recipients** to choose one or more email recipients for the notification.

   You can select existing users, manually enter one or more email addresses, or use variables to add recipients dynamically.

   The **Recipients** field supports variables when **Run for each record** is enabled. However, it doesn't support variables when it's disabled.

   To add a new recipient, see Adding a Recipient.
5. (Optional) Edit the **Subject**.

   The subject can be plain text or it can be a template in either Mustache or Jinja2. For example:

   Mustache

   ```
   Alert: {{ctx.payload.filtered_total}} events detected
   ```

   Jinja2

   ```
   Security Alert: {{ ctx.payload.filtered_total }} events require review
   ```

For information about using templates for email subjects, see **Email Action Templates**.

6. Set the priority for the email message as **Low**, **Normal**, or **High**.
7. Select **Include Interflow** to have the Interflow data appended to the email.
8. If you added a calculation when building a query, select **Include CSV of Calculation Results**, select **Include JSON of Calculation Results**, or select both to attach a CSV file, a JSON file, or both types of files to the email.

   CSV Files Included with Calculation Results

   If you have multiple calculations, the email will include multiple attachments with the results of each calculation in its own CSV file.

   6.1.0 From 6.1.0, if you have a single calculation, Stellar Cyber attaches two CSV files to a notification email. If you have multiple calculations, the email includes multiple pairs of attachments, with the results of each calculation in its own set of CSV files.

   The pair of CSV attachments includes the following:

   - Original CSV – Contains the complete calculation results before any condition filtering.
   - Filtered CSV – Contains only the rows that meet the rule condition. This file is named filtered_<layer1>_<layer2>.csv.

   This change addresses cases where rules use the **All** or **At Least One** conditions. Previously, only the unfiltered results were attached, so it was not always clear why an **All** condition did not trigger while an **At Least One** condition did. With the filtered CSV, you now see exactly which rows met the condition, alongside the original unfiltered results for context.

   For example, suppose a rule condition requires a count greater than or equal to 5. In this case, the **At Least One** condition is satisfied because at least one row meets the threshold, and the filtered CSV shows those qualifying rows. Meanwhile, the **All** condition does not trigger because not every row meets the threshold, which is evident by comparing the filtered CSV against the original CSV.

   Example

   Original CSV (srcip_dstip.csv)

   ```
   srcip,dstip,count

   10.1.1.1,172.16.1.10,2

   10.1.1.1,172.16.1.11,7

   10.1.1.2,172.16.1.20,9
   ```

   Filtered CSV (filtered_srcip_dstip.csv)

   ```
   srcip,dstip,count

   10.1.1.1,172.16.1.11,7
   ```

```
10.1.1.2,172.16.1.20,9
```

Providing both files ures you can see the complete calculation results for context and a filtered view of only the rows that matched the configured condition.

JSON Files Included with Calculation Results

The JSON file contains the same filtered calculation results as the CSV attachment—only records that meet the condition threshold of the calculation—but in a structured, machine-readable format. Use this option when you want to work with the results programmatically. You can pass the JSON directly to a script, automation, or downstream tool to parse or integrate the calculation results, instead of reviewing them only as a CSV spreadsheet.

9. (Optional) Edit the **Email Body**.

   The **Email Body** field can contain plain text, a template, or a combination of plain text and template expressions. For example, you might enter a plain-text statement such as `AWS CloudTrail alerts need your attention.` followed by a template expression such as `Alert: {{ctx.payload.filtered_total}} events were detected.` Variables that resolve successfully are replaced with their values, and variables that cannot be resolved render as empty text rather than generating an error. You can also leave the Email Body field empty, in which case the email is sent without a message body.

   For details about using a template in the email body, see **Email Action Templates**.

   To prevent malicious content from being stored in playbooks and sent in outbound emails, Stellar Cyber sanitizes HTML in the Email Body field when you create or edit an ATH playbook. Stellar Cyber removes unsupported HTML tags and attributes. For example, `<img>` tags are dropped, outer wrapper tags such as `<!DOCTYPE>`, `<html>`, `<head>`, and `<body>` are stripped, and legacy table layout attributes such as `width`, `cellpadding`, `cellspacing`, `bgcolor`, `align`, `valign`, and `height` are removed. Inline `style=` attributes are preserved only on these tags: `span`, `div`, `p`, `table`, `td`, and `th`. Inline styles on tags such as `a`, `b`, `tr`, `hr`, `thead`, `tbody`, and `tfoot` are removed. In `href` attributes, only `http`, `https`, and `mailto` URL schemes are allowed. Existing saved email bodies are not changed until the playbook is edited.

   For reference, the following is the complete preserved-tag list: `b`, `i`, `strong`, `em`, `u`, `br`, `p`, `ul`, `ol`, `li`, `a`, `h1–h6`, `table`, `tr`, `td`, `th`, `thead`, `tbody`, `tfoot`, `span`, `div`, `blockquote`, `pre`, `code`, and `hr`. Anything not on this list—such as `img`, `script`, `style`, `iframe`, `font`, `center`, and so on—is removed.

10. (Optional) Select **Override Default Settings** for **Mute** to suppress this email action for a defined duration. Then specify the time interval to prevent the action from executing.

    If the ATH playbook is muted, Stellar Cyber does not generate alerts or execute any actions, regardless of the action-level mute configuration. For more information, see **Muting Actions**.

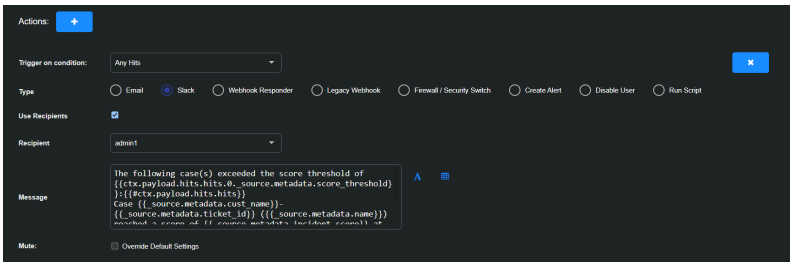
Slack Action

Before you add a Slack ac, make sure you enable incoming webhooks in Slack and create an incoming webhook for use with Stellar Cyber. Refer to this article in the Slack API documentation for details.

To configure a Slack message as an action:

1. Click the + button to add an action.

   Another action appears below the existing action.



2. Select the **Trigger on condition**.
3. Select **Slack** for the **Type**.
4. If the recipient is already in Stellar Cyber, select **Use Recipients** and select the recipients for the message.
5. If the recipient is not in Stellar Cyber, enter the **URL** you created in Slack for incoming webhooks.

   That incoming webhook is associated with the Slack channel where the message will be posted.

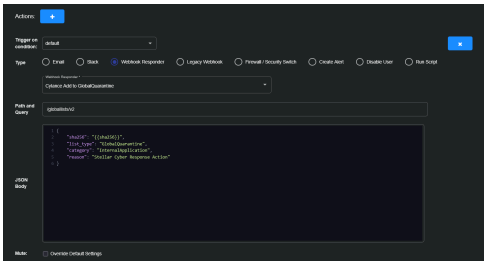
6. Add the message to be sent.

Webhook Responder Action

When you configure a Webhook Responder, you can use it in an ATH playbook. You can select the Webhook Responder to use for the action triggered by ATH.

To create a webhook responder rule for an action:

1. Select **Automation | Automation** and then select **Create**.
2. Configure the ATH playbook and scroll down to **Actions**.



3. Select the + button to add an action.

   Another action appears below the existing action.

4. Select the **Trigger on condition**.
5. Select **Webhook Responder** for the **Type**.

6. Choose a previously configured Custom Webhook, Acronis, Cylance, ESET, Netskope, Proofpoint on Demand, Trend Vision One, WatchGuard Firebox, or WithSecure Elements Responder from the drop-down menu.

   The **Path and Query** and **JSON Body** fields populate.

7. (Optional) Modify the **Path and Query** or **JSON Body**.

   When you select a preconfigured Custom Webhook Responder, the system prepopulates the JSON Body field with a structure designed to match the expectations of the receiver. You can modify this body to include additional fields if needed.

   When you do modify it, use direct field references with Mustache syntax. Do not use iteration tags such as `{{#ctx.payload.filtered}}` and `{{/ctx.payload.filtered}}`. Webhook Responders process one result at a time and do not require iteration. The system automatically flattens field names, so you do not include `_source.` in any field reference.

   The Webhook Responder differs from other playbook actions that operate on a list of results. This action triggers once per result, typically one alert at a time, such as when used from the Alerts page. Because Stellar Cyber already loops through each record automatically, you do not iterate over `ctx.payload.filtered`. Instead, reference fields from the current record directly using Mustache syntax, like `{{dstip}}` or `{{stellar_uuid}}`.

   You can reference event fields such as `{{dstip}}`, `{{dstip_host}}`, `{{appid_name}}`, and `{{metadata.request.server_name}}`, and enriched fields such as `{{_id}}`, `{{_index}}`, and `{{stellar_uuid}}`.

   The system deduplicates webhook requests. If multiple results render the same JSON body, the responder sends only the first and skips the rest.

   Example: Basic Custom Format

   Use this format when defining a simple JSON payload:

```
{
  "DST-IP": "{{dstip}}",
  "Host": "{{dstip_host}}",
  "Reputation": "{{dstip_reputation}}",
  "Link": "https://your-
tenant.stellarcyber.cloud/alerts/alert/{{_index}}/_doc/{{_id}}",
  "UniqueID": "{{stellar_uuid}}"
}
```

   Example: Adding Fields to a Prepopulated Ticket Structure

   If the prepopulated JSON defines a helpdesk ticket structure, you can insert additional event data into the comment body:

```
{
  "ticket": {
    "comment": {
      "body": "Alert link: https://poc-
eu.stellarcyber.cloud/alerts/alert/{{_index}}/_doc/{{_id}}. Unique ID:
{{stellar_uuid}}. DST-IP: {{dstip}}. Host: {{dstip_host}}. Reputation:
{{dstip_reputation}}."
    },
    "priority": "urgent",
    "subject": "Criticality: Critical {{xdr_event.display_name}}"
  }
}
```

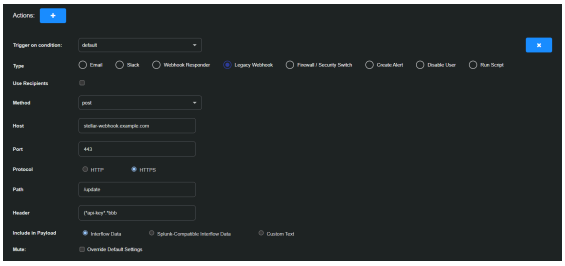
Use this format when enriching a service ticket with alert details while preserving the expected structure of the receiving system.

8. Select the **Override Default Settings** option to override the playbook mute settings and mute just this action.
9. Select **Submit**.

Legacy Webhook Action

To create a legacy webhook rule for an action:

1. Click the ✚ button to add an action. Another action appears below the existing action.



2. Select the **Trigger on condition**.
3. Select **Legacy Webhook** for the **Type**.
4. Select **Use Recipients** if the recipient is already in Stellar Cyber, if desired and applicable for the webhook. Select the **Recipient** if you do so.

   Keep in mind the following rules for webhook usernames and passwords:

   - For versions prior to 5.03:
     - Enter the **username** without the **@** or period **(.)** characters.
     - Enter the **password** without any non-alphanumeric characters except for the underscore (**_**).
   - From v5.0.3 and later, you can use any non-alphanumeric characters in both the username and password fields, including the **@** and period **(.)** characters.
5. Choose the **Method** (Put or Post).
6. Specify the **Host** the webhook is accessing. Do not include the http scheme; that is specified below.
7. Specify a mandatory **port** number.
8. Select whether the method needs to use **HTTP** or **HTTPS**.
9. Provide the **Path** to the webhook action.
10. For the **Header** field, enter the header the webhook action expects.
11. Select the data you want to package with the webhook (**Interflow, Splunk-compatible Interflow**, or **Custom Text**.

    If you select **Custom Text** message, enter the **Body** of the text. You can also use the **Select a Variable** and **Define a Result Table** options to help populate the **Body** of the text. Keep in mind, however, that most webhook destinations expect valid JSON and that the **Define a Result Table** simply creates a body with one line of text per hit. You

must insert {""} characters as necessary to create valid JSON from the text created by the **Define a Result Table** feature:

- The entire body of the text must be enclosed in { }.
- The values must be between {{#ctx.payload.filtered}} and {{/ctx.payload.filtered}} tags.
- Commas must be used to separate lines.
- The body is formatted using Mustache Templating.

Here is an example of properly formatted JSON for the Body:

```
{"a":"{{ctx.payload.filtered_total}}",
"b":"{{ctx.payload.hits.total.value}}",
"c" : " test",
{{#ctx.payload.filtered}}  "appid_name": "{{_source.appid_name}}",
{{/ctx.payload.filtered}} "d": " {{ctx.trigger.scheduled_time}}"}
```

This code creates the following:



12. Specify whether you want to override the playbook's mute settings and mute just this action.
13. Select **Submit**.

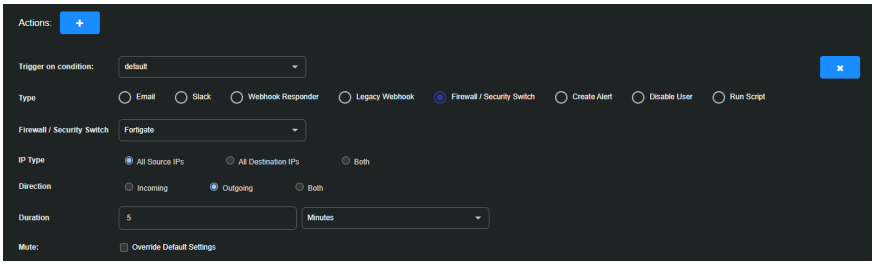
Firewall / Security Switch Action

To create a firewall rule as an action:

> When you create a correlation for use in a Firewall action, **Grouping** must be toggled OFF in the correlation query editor. In addition, that query must include `srcip` and `dstip` for the host on which you want to perform the Firewall action. You must ensure that the Active box is enabled for this same host in the list displayed below the connector selection menu.

1. Click the ✚ button to add an action.

   Another action appears below the existing action.



2. Select the **Trigger on condition**.
3. Select **Firewall / Security Switch** for the **Type**.

4. Select the desired connector from the **Firewall / Security Switch** dropdown.

   The query objects with [redacted]t addresses are listed below with an **Active** check box.

5. Enable the **Active** check box to make the addresses from that query object active for response actions. You may see more than one query object listed here. If so, you need only enable the **Active** boxes that include the extracted `srcip/dstip` on which you want to perform the action.

6. Adjust the directional selectors needed, depending on the type of firewall you choose, the IP address/traffic configuration options are automatically adjusted. For example, with Meraki Firewall actions, the **Incoming** and **Both** fields are disabled for **Direction** because the firewall rules that Stellar Cyber enforces on Meraki firewalls always apply to **Outgoing** traffic.

7. Set a duration for the rule.

Create Alert Action

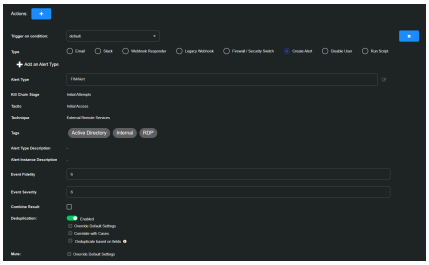
If you configure a playbook with the **Create Alert** action and add calculations, make sure the scope is set to a single tenant. If the scope is **All Tenants** with multiple tenants, alerts aren't created because Stellar Cyber can't assign a valid tenant ID to them. Other actions, including email notifications and webhook responders, still work because they don't require a tenant ID.

You can either select an existing alert type from the **Alert Type** dropdown or create a new one using the procedure below.

If you drilled to the ATH Playbook workflow from a custom alert type in the Detections | Alerts page, the **Type** is already set to **Create Alert** and the **Alert Type**, **Kill Chain Stage**, **Tactic**, **Technique**, and **Tags** fields are all pre-filled based on the custom alert type settings. Refer to **Creating New Alert Types** for details.

1. Click the ➕ button to add an action.

   Another action appears below the existing action.



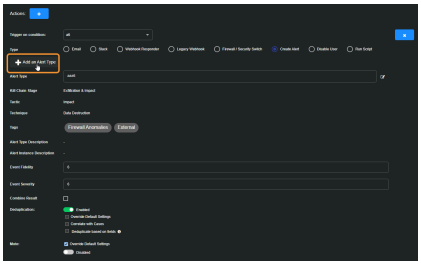
2. Specify the **Trigger on condition**.

3. Set the **Type** to **Create Alert**.

4. Specify the **Alert Type**.

   If you launched the ATH Playbook workflow from a custom alert type in the Alerts list, you do not need to perform this step. These settings are already configured based on the custom alert type from which you launched the workflow.

You can either select an existing alert type from the dropdown or add a new one by clicking the **Add an Alert Type** button to launch the standard workflow to add a new custom alert type.



5. Enter the **Event Fidelity** and **Event Severity**.

   Make sure that you configure an **Event Fidelity** greater than or equal to **1** for alerts triggered by the playbook to appear in the Alerts list. Alerts with an **Event Fidelity** of zero are not listed in the Alerts display.

6. Click **Combine Result** to combine multiple records in that time period into a single event. This reduces the number of events generated for a single actual event.

   Refer to **How Deduplication Works with the "Combine Result" Option** for details on the interaction between these two options.

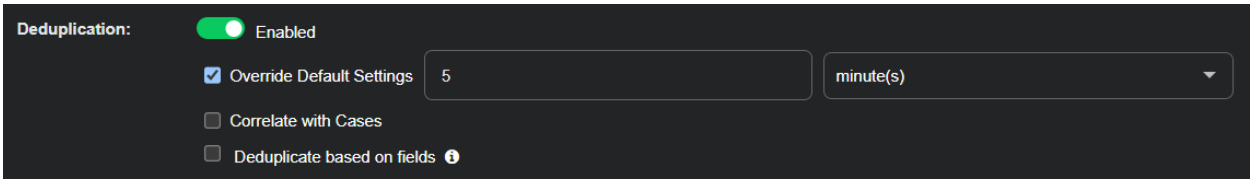
7. Use the **Deduplication** options to specify how multiple instances of this alert within a small time window are deduplicated.

   To reduce noise and focus attention, by default, Stellar Cyber deduplicates multiple instances of the same custom alert that fire within the rule's **Run every** time window (that is, from the time the rule ran last to the time of its current iteration).

   As illustrated below, you can set the following options for deduplication of custom alerts:

   - Toggle the feature on (**Enabled**) or off (**Disabled**). By default, it is enabled.
   - Use the **Override Default Settings** option to specify a custom time window within which multiple instances of the same alert are deduplicated.

     By default, custom alerts are deduplicated within a rule's **Run every** time window. The **Override Default Settings** options let you specify a custom time window.



Stellar Cyber performs this deduplication by discarding fields that are always different between separate alerts, calculating a hash, and monitoring future events for matches.

Refer to **About Deduplication of Custom Alerts** for details on which fields are not considered for deduplication, how to identify deduplicated alerts in JSON and

how the deduplication timer works together with both the **Run every** setting for the ATH Playbook as a whole and the **Combine Result** option.

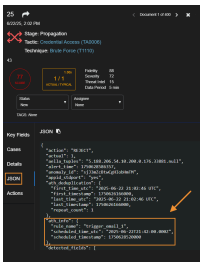
- Check the **Correlate with Cases** option if you would like Stellar Cyber to create a new case based on the alert.
- Check the **Deduplicate based on fields** option if you would like to consolidate records based on specific matching fields. Once you enable this option, a dropdown appears where you can select the specific fields to use for deduplication. You can also type in your own custom fields manually, including nested fields by adding a period between fields (`field.subfield`, for example).

This option is useful in situations where a particular connector is sending a new alert record every time an alert is updated on the connector source. You can use it together with the **Override Default Settings** option to consolidate records with specified matching fields within a given window.

When choosing fields for deduplication, choose fields that are not likely to be unique between records. For example, timestamps are very likely to have different values in different records. If you try to deduplicate records based on a timestamp, you are not likely to find matching records to consolidate. This deduplication option is supported with all Stellar Cyberindexes.

Viewing Deduplication Information in Alert Displays

You can add the **ATH Deduplication Last Time UTC** column to an Alerts table to see the timestamp of when an alert record was deduplicated. In addition, the JSON for a consolidated alert record includes an **ath_info** section with details on the deduplication, including the name of the ATH Playbook that produced the consolidated record and when it took place. For example:



Specifying Multiple Fields for Deduplication

Click here to see details on how Stellar Cyber handles situations where multiple fields are specified for deduplication.

If you specify multiple fields for the **Deduplicate based on fields** option, they are joined with a logical **AND** when looking for matched records to deduplicate. If specified fields are present in multiple records, they must all match in order for records to be deduplicated. Not all fields need to be present in both records, but the ones that are there must match in order to be deduplicated.

Consider a sample scenario where we have defined the following three fields for the **Deduplication based on fields** option: `[field_1, field_2, field_3]`.

All Records Present and Matched

In our first example, we have two records, `Record X` and `Record Y`, each with matching values for all three of these fields. In this case, the two records will be deduplicated and consolidated to a single record:

```
Record_X = {
  "field_1": "value_1",
  "field_2": "value_2",
  "field_3": "value_3",
  "other_field": "other_value_1",
  "timestamp": 11111111,
}

Record_Y = {
  "field_1": "value_1",
  "field_2": "value_2",
  "field_3": "value_3",
  "other_field": "other_value_2",
  "timestamp": 22222222,
}
```

Partial Matches Across Three Records

In our second example, we have three records – `Record X`, `Record Y`, and `Record Z`:

```
Record_X = {
  "field_1": "value_1",
  "other_field": "other_value_1",
  "timestamp": 11111111,
}

Record_Y = {
  "field_1": "value_1",
  "other_field": "other_value_2",
  "timestamp": 22222222,
}

Record_Z = {
  "field_1": "value_1",
  "field_2": None,   # <-----------
  "other_field": "other_value_2",
  "timestamp": 22222222,
}
```

In this example, the records are deduplicated as follows:

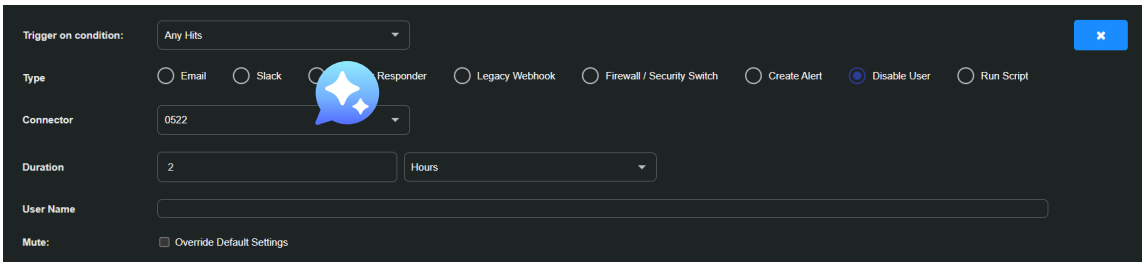
- `Record X` and `Record Y` both have identical values for `field_1` (`value_1`). In addition, `field_1` is the **only** field from our list that appears in the two records. Because of this, they are consolidated to a single record.
- `Record Z` also has the same value as the first two records for `field_1` (`value_1`). However, because `field_2` is also present in `Record Z` and is not the same as either `Record X` or `Record Y`, it is not deduplicated.

8. Specify whether you want to override the playbook's mute settings and mute just this action.

Disable User Action

To disable a user as an action:

1. Click the ➕ button to add an action. Another action appears below the existing action.



2. Specify the **Trigger on condition**.
3. Select **Disable User** for the **Type**.
4. Select the Active Directory connector or the Microsoft Entra ID connector for the action from the drop-down list.
5. Set how long you want to disable the user in the **Duration** field.

   You can choose **Minutes**, **Hours**, **Days**, or even **Forever**.
6. Use the **User Name** field to specify one or more users to block. You can specify users with a combination of the following techniques:
   - Select one or more users from the prepopulated dropdown list.
   - Type a **username** or **objectSID** directly into the **User Name** field.

     An **objectSID** typically appears in a format similar to the following example: `S-1-5-21-1068523723-946037408-1232828536-51066`

   Note: You can only block users in the same domain as the selected Active Directory or Microsoft Entra ID connector. Check the success of User Disable actions in the **User Actions** tab of the Automation | Action History.
7. Specify whether you want to override the mute settings in the playbook and mute just this action.

Script Action

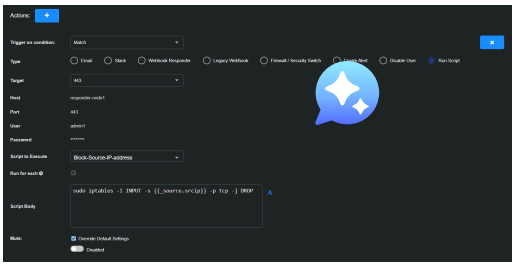
A script action is an automation task in an ATH playbook that runs a custom script when defined conditions are met. Script actions extend playbook capabilities by allowing tailored responses or integrations beyond built-in actions.

The following instructions explain how to configure and manage script actions, including guidelines for script preparation and execution. For a detailed ATH playbook configuration involving a script, see **ATH Example: Reset TCP Connections after Detecting Threats**.

To run a script via SSH as an action:

1. Select the ✚ button to add an action.

   Another action appears below the existing action.



2. Select the **Trigger on condition**.
3. Select **Run Script** for the **Type**.
4. Select the **Target** from the dropdown.

   Stellar Cyber fills in the details for that target. The target must be a remote host already in Stellar Cyber. If not, you can create a new remote host.
5. Select the **Script to Execute**.

   The script is automatically entered in the **Script Body**. The script must already be in Stellar Cyber. If not, you can create a new script.
6. Select **Run for each** if you want the script to run for each hit the playbook finds. If there are many hits, running the script for each of them can take a significant amount of time. Leave this unselected to run only once each time the playbook runs.

   Do not select **Run for each** if your script uses variables for the execution time (`ctx.execution_time`) or variables that include the results of an executed query such as `ctx.payload.filtered_total` or `ctx.payload.hits.total`. However, it's safe to select **Run for each** if your script uses any other types of variables within an Interflow record.

   If your script produces an event list with multiple results and an action instance for each result, it's important to select **Run for each**. If it's not selected, the script renders all results in a single action and Stellar Cyber only runs that action, which might not be the one you want for all results. However, when you select **Run for each**, Stellar Cyber runs an individualized action for each and every result.

   In addition, when **Run for each** is enabled, it's unnecessary to include start and end instructions in the script; that is, `{{#ctx.payload.filtered}}` for starting an iteration and `{{/ctx.payload.filtered}}` for ending an iteration. For example, instead of this —

   ```
   {{#ctx.payload.filtered}}
   block_one_ip {{srcip}}
   {{/ctx.payload.filtered}}
   ```

   —the script only needs this single statement:

   ```
   block_one_ip {{srcip}}
   ```
7. You can edit the **Script Body**.
8. Select **Submit**.

The playbook is added. Stellar Cyber automatically enables the new playbook and displays the playbook list.

Muting Actions

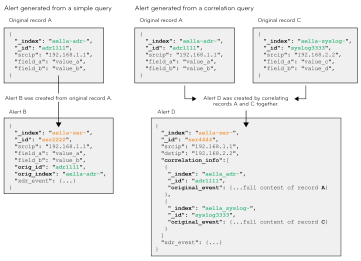
You can use the **Mute** feature in the Actions panel to temporarily halt specific individual actions associated with a playbook. Select **Override Default Settings** and toggle **Mute | Disabled** to **Enabled**. Then use the adjacent fields to specify for how long you would like to stop the action from triggering.

You can also mute alerts associated with the playbook. Note that Alert level mutes take precedence over Action level mutes. If the alert is muted, no actions will be triggered. If the alert is unmuted, you can mute individual actions.

Trace an Alert to Its Underlying Event Records

If you want to trace an alert back to the original events that triggered it, the alert record includes reference fields and values that allow you to locate the source records. The method differs depending on whether the alert was generated by a simple query or a correlation query.

- **Simple query alerts**: Original records can be found through the values of the `orig_id` and `orig_index` fields.
- **Correlation query alerts**: Original records can be found through the values of the `correlation_info:_id` and `original_event` fields.
- In both cases, the alert record provides a reference to the `_id` value of the records of the original events that triggered the alert.



Finding the original records for an alert generated by a simple query:

1. Search for the alert in the Alerts index using its `_id` value.
2. Check the alert details in the search result.

   If the alert was generated by a simple query, locate the `orig_id` and `orig_index` fields.
3. Use `orig_id` and `orig_index` to find the original event record:

   `orig_id` contains the `_id` value of the original record.

   `orig_index` indicates which index the original record is stored in.
4. Perform a new search in the specified index (`orig_index`), but replace `orig_id` with `_id`.

Example:

1. Search the Alerts index for an alert with `"_id": "ser2222"`.

   In the alert details, you find:

   `"orig_id": "adr1111"`

`"orig_index": "aella_adr-"`

This tells you that ID of the original event record is adr1111 (`"_id": "adr1111"`) and is stored in the Traffic index (`aella_adr-`).

2. Search the Traffic index for `"_id": "adr1111"` to retrieve the full details of the original event.

Finding the original records for an alert generated by a correlation query:

1. Search for the alert in the Alerts index using its `_id` value.
2. Check the alert details in the search result.

   If the alert was generated by a correlation query, note the values of the `index`, `_id`, and `original_event` fields for each related event in the `correlation_info` array.
3. Find the original records using one of two methods:

   Search for the `_id` values in the specified index for each related event.

   or

   Check the `original_event` field, which contains the full content of the correlated event record.

This structure ensures you can efficiently investigate alerts by accessing their source data, whether they were identified by a single query match or multiple correlated events.

About Deduplication of Custom Alerts

Stellar Cyber deduplicates custom alerts by discarding fields that are always different between separate alerts, calculating a hash, and monitoring future events for matches. Because the following fields are always different between separate records, they are not considered when calculating the hash used for deduplication:

Exact matches between:

- stellar_index_id
- stellar_uuid
- timestamp
- timestamp_utc
- write_time
- xdr_event
- index
- id
- type

Fields containing the following substrings:

- stellar
- time
- date

- index

Identifying Deduplicated Alerts

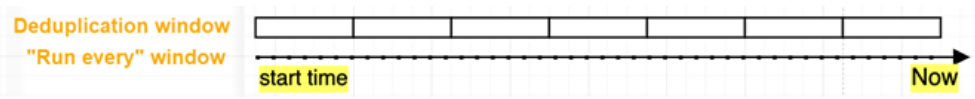
If an alert has been deduplicated, there are additional fields in its JSON record indicating that the document represents multiple alerts firing repeatedly over a period of time. The key field to look for is `ath_deduplication`. For example:

```
"ath_deduplication": { // <-- if this field exists, the alert has been deduplicated
  "first_time_utc": "2023-01-25 07:34:49 UTC",
  "first_timestamp": 1674632089999,
  "last_time_utc": "2023-01-25 07:59:50 UTC",
  "last_timestamp": 1674633590521,
  "repeat_count": 6 // <-- repeat count in the time window ^
},
```

This record indicates that there have been six identical alerts in the time window. Note that Stellar Cyber continues to update this document if new instances of the same alert arrive in the time window. You can also see the timestamps for the first and last identical alerts represented by this document.

How the Deduplication Interval Works with the "Run every" Setting

When you configure an ATH Playbook, you supply a **Run every** setting that specifies how often the rules in the playbook run. The two settings work together as follows:

Settings of "Run every" and Deduplication Time Window	Results
"Run every" is greater than deduplication time window	Deduplication is performed over the configured time window. Because the deduplication window is smaller than the "run every" window, that means there can be multiple alerts over the "run every" interval. In this case, because alerts are deduplicated over a time window that is smaller than the ATH Playbook's "run every" interval, only one identical alert per deduplication window is created. For example, suppose an ATH Playbook is configured to run once a week with a deduplication time window set to one day. As you can see below, the alerts in each of the deduplication time windows (represented by dots on the timeline) would be reduced to a single identical alert per deduplication window. In this example, that would mean a total of seven alerts for the "run every" interval of one week: 
"Run every" is less than the deduplication time window	In this case, because alerts are deduplicated over a time window that is greater than the ATH Playbook's "run every" interval, only one identical alert per deduplication window is created.
"Run every" and deduplication time window are identical	This is the default configuration and results in only a single identical alert per "run every" interval.

How Deduplication Works with the "Combine Result" Option

You can enable both the **Combine Result** and **Deduplication** options for a playbook. Keep in mind the following if you do so:

- The **Combine Result** option has a higher priority than Deduplication. Deduplication is not performed until after results are combined.
- Because of this, if both options are enabled, deduplication has no effect when the deduplication window is less than or equal to the **Run Every** window. This is because the **Combine Result** feature ensures that there is only a single alert document in each **Run Every** window.
- Looked at another way, if you enable both **Combine Result** and **Deduplication**, then deduplication will only take place if the deduplication window is greater than the **Run Every** window. In this case, the deduplication window cover multiple iterations of the rule, meaning that there would potentially be multiple combined alerts in the deduplication window to be deduplicated.

© 2026 Stellar Cyber . All rights reserved.